



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 07 - RAG COM VECTOR DATABASE NA
PRÁTICA



OBJETIVOS DA AULA

- ❑ Configurar um Vector Database (PgVector) utilizando o Quarkus Dev Services.
- ❑ Substituir a dependência do EasyRAG por uma configuração explícita de um pipeline RAG.
- ❑ Implementar um serviço de ingestão de dados programático que é executado na inicialização da aplicação.
- ❑ Criar um RetrievalAugmentor conectado o AiService ao EmbeddingStore persistente.



MÃOS À OBRA

VECTOR DATABASE VIA QUARKUS DEV SERVICES

- ❑ Tarefa
 - ❑ Configurar um banco de dados para desenvolvimento local pode ser tedioso (instalar, configurar, iniciar/parar).
- ❑ Solução via Quarkus
 - ❑ O Dev Services automatiza o provisionamento da instância durante o desenvolvimento.
 - ❑ Ao adicionar uma extensão de banco de dados, o Quarkus detecta se uma instância está rodando. Se não encontrar, ele automaticamente cria um container pronto para uso.
- ❑ Passo 1: Adicionar a Dependência do PgVector
 - ❑ Vamos usar o PgVector, uma extensão para o popular banco de dados PostgreSQL. Adicione a extensão ao seu projeto com o comando:
 - ❑ `mvn quarkus:add-extension -Dextensions="quarkus-langchain4j-pgvector"`
 - ❑ Apenas por adicionar esta dependência, o Quarkus agora gerenciará um container PostgreSQL com a extensão PgVector habilitada para nós em modo de desenvolvimento.

SUBSTITUINDO O EASYRAG

- ❑ Solução mais "parruda"
 - ❑ O EasyRAG foi ótimo para começarmos a entender o RAG, mas agora queremos algo mais próximo da produção.
- ❑ Passo 1: Remover a Dependência do EasyRAG
 - ❑ Use a o comando do Maven para remover a extensão do seu pom.xml:
 - ❑ `mvn quarkus:remove-extension -Dextensions="quarkus-langchain4j-easy-rag"`
- ❑ Passo 2: Desativar a Configuração Anterior
 - ❑ Abra o arquivo `src/main/resources/application.properties` e comente as propriedades relacionadas ao EasyRAG para desativá-las.

```
# --- Configurações do EasyRAG (desativadas) ---  
# quarkus.langchain4j.easy-rag.path=src/main/resources/rag  
# quarkus.langchain4j.easy-rag.max-segment-size=100  
# quarkus.langchain4j.easy-rag.max-overlap-size=20
```

CONFIGURANDO O PGVECTOR

- ❑ Conectando ao Vector Database
 - ❑ Agora, precisamos configurar nossa aplicação para usar o PgVector como EmbeddingStore e definir nosso modelo de embedding.
- ❑ Certifique-se de que o modelo nomic-embed-text foi baixado no seu servidor Ollama com o comando
 - ❑ `ollama pull nomic-embed-text`
- ❑ Adicione as seguintes propriedades ao seu `application.properties`:

--- Configuração do Embedding Model ---

`quarkus.langchain4j.ollama.embedding-model.model-id=nomic-embed-text`

--- Configuração do PgVector ---

A dimensão do vetor DEVE corresponder à do seu EmbeddingModel. O modelo 'nomic-embed-text' usa vetores de 768 dimensões.

`quarkus.langchain4j.pgvector.register-vector-pg-extension=true`

`quarkus.langchain4j.pgvector.dimension=768`

`quarkus.langchain4j.pgvector.table=travel_embeddings`

`quarkus.langchain4j.pgvector.drop-table-first=true`

INGESTÃO DE DADOS

- ❑ Vamos criar um serviço que carrega, processa e armazena nossos documentos no PgVector quando a aplicação inicia.

```
package dev.ia;

[IMPORTS...]

@ApplicationScoped
public class DocumentIngestor {

    @Inject
    EmbeddingStore<TextSegment> store;

    @Inject
    EmbeddingModel embeddingModel;

    public void onStart(@Observes StartupEvent event) {
        Document document = FileSystemDocumentLoader.loadDocument(
            Paths.get("src/main/resources/rag/pacotes-viagem.md")
        );

        document.metadata().put("type", "packages");

        DocumentSplitter splitter = DocumentSplitters.recursive(200, 20);

        EmbeddingStoreIngestor ingestor = EmbeddingStoreIngestor.builder()
            .documentSplitter(splitter)
            .embeddingModel(embeddingModel)
            .embeddingStore(store)
            .build();

        ingestor.ingest(document);
    }
}
```

CONFIGURANDO O RETRIEVAL AUGMENTOR

- ❑ Como removemos o EasyRAG, precisamos dizer ao nosso AiService como encontrar os dados. Fazemos isso criando nosso próprio RetrievalAugmentor como um bean CDI.

```
package dev.ia;

import dev.langchain4j.model.embedding.EmbeddingModel;
import dev.langchain4j.rag.DefaultRetrievalAugmentor;
import dev.langchain4j.rag.RetrievalAugmentor;
import dev.langchain4j.rag.content.retriever.EmbeddingStoreContentRetriever;
import dev.langchain4j.store.embedding.EmbeddingStore;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.inject.Produces;
import dev.langchain4j.data.segment.TextSegment;

@ApplicationScoped
public class RagConfiguration {

    @Produces
    public RetrievalAugmentor retrievalAugmentor(EmbeddingStore<TextSegment> embeddingStore, EmbeddingModel embeddingModel) {
        return DefaultRetrievalAugmentor.builder()
            .contentRetriever(EmbeddingStoreContentRetriever.builder()
                .embeddingStore(embeddingStore)
                .embeddingModel(embeddingModel)
                .maxResults(5)
                .build())
            .build();
    }
}
```

REFINANDO O AISERVICE

- ❑ Vamos criar um AiService mais focado e com uma persona bem definida.

```
package dev.ia;

import dev.langchain4j.service.MemoryId;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import io.quarkiverse.langchain4j.RegisterAiService;

@RegisterAiService
public interface PackageExpert {

    @SystemMessage("""
        Você é um assistente virtual da 'Mundo Viagens', um especialista em nossos pacotes de viagem.
        Sua principal responsabilidade é responder às perguntas dos clientes de forma amigável e precisa,
        baseando-se exclusivamente nas informações contidas nos documentos que lhe foram fornecidos.
        Nunca invente informações ou use conhecimento externo.
        Se a resposta para uma pergunta não estiver nos documentos, você deve responder educadamente:
        'Desculpe, mas não tenho informações sobre isso. Posso ajudar com mais alguma dúvida sobre nossos pacotes?'
        """)
    String chat(@MemoryId String memoryId, @UserMessage String userMessage);
}
```

REFINANDO O AISERVICE

- ❑ Atualize o TravelAgentResource.java no pacote dev.ia para usar este novo especialista.

```
package dev.ia;

import jakarta.inject.Inject;
import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/travel")
public class TravelAgentResource {

    @Inject
    PackageExpert expert;

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String ask(String question) {
        return expert.chat("session-123", question);
    }
}
```

TESTANDO A NOVA ARQUITETURA

- ❑ Passo 1: Iniciar a Aplicação
 - ❑ `mvn quarkus:dev`
 - ❑ Na primeira vez, o DocumentIngestor será executado e populará o banco.
- ❑ Passo 2: Testar a Busca
 - ❑ Faça uma pergunta para validar que o RAG continua funcionando:

```
curl -X POST -H "Content-Type: text/plain" \  
-d "Qual a politica de cancelamento para o pacote Aventura Amazonia?" \  
http://localhost:8080/travel
```

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ O que Construímos Hoje?
 - ❑ Integramos um Vector Database (PgVector) de forma transparente com os Dev Services do Quarkus.
 - ❑ Removemos a abstração do EasyRAG e assumimos o controle total do nosso pipeline.
 - ❑ Criamos um serviço de ingestão programático que carrega, enriquece com metadados e armazena nossos documentos de forma persistente.
 - ❑ Configuramos o RetrievalAugmentor manualmente, entendendo como as peças do RAG se conectam.
- ❑ Próxima Aula:
 - ❑ Até agora, nosso agente apenas responde a perguntas. Na sequência, vamos dar o próximo grande passo: evoluir de um chatbot de Q&A para um Agente Inteligente capaz de executar ações no mundo real. Aprenderemos como o LLM pode usar "Tools" (nossos métodos Java) para consultar bancos de dados, chamar APIs e realizar tarefas complexas em nome do usuário.

ATÉ A PRÓXIMA!